Gliwice, 23.06.2025

Biologically inspired artificial intelligence

Report

Topic:

Recognizing F1 cars from photos

Supervisor:

Dr. Inż. Grzegorz Baron

Section:

Oskar Chłopaś

Piotr Nowak

1. Introduction

The goal of the project was to create a deep learning model capable of recognizing F1 race cars in images. The project code allows training such a model, saving it, evaluating its performance, and using it to classify new images. Transfer learning was utilized with several neural networks, and a dataset containing images of various Formula 1 teams was used.

2. Analysis of the task

There are many deep learning libraries that are implementing CNN. For our project we chose TensorFlow. It provides high-level API (Keras) which is easy to learn and it is much more readable. It also provides pre-trained models, which we used for many experiments we performed with them.
One of the cons of this approach is that TensorFlow provides less control over the models we build. Libraries like PyTorch are more low-level which makes them more flexible in parameters we want in our model.

As input data, we used a database of nearly 4,000 images of F1 cars, found on the Kaggle repository. It has 8 different classes containing various F1 cars such as: AlphaTauri, McLaren Red Bull etc. Unfortunately, some of the images were corrupted, as TensorFlow couldn't read some formats of the images in this dataset. This issue was later fixed.

3. Libraries

TensorFlow – An open-source machine learning library created by Google. It is responsible for, among other things:

- Building the convolutional neural network (CNN) model,
- Training and optimizing the model's weights,
- Predicting classes for images.

- tensorflow.keras.applications
  Provides ready-made CNN architectures. These models are often pre-trained on the ImageNet dataset, which allows the use of transfer learning — that is, transferring already learned knowledge to a new task.
- Seaborn – for creating heatmap in confusion matrix
- Numpy – for example for choosing the class with the highest probability
- Matplot – for creating plots of model accuracy and loss

4. **Code**

Imports and environment setup

```
import os
os.environ['TF_ENABLE_ONEDNN_OPTS'] = '0'
import tensorflow as tf
import matplotlib.pyplot as plt
import json
import numpy as np
import seaborn as sns
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.applications import MobileNetV2
from tensorflow.keras.applications import NASNetMobile
from tensorflow.keras.applications import ResNet50V2
from tensorflow.keras import layers, Model, Input
from tensorflow.keras.models import load_model
from PIL import Image
```

Here we import all the required libraries for data handling (os, json, numpy, PIL), deep learning (tensorflow, keras), image augmentation (ImageDataGenerator), and visualization (matplotlib, seaborn). The environment variable TF_ENABLE_ONEDNN_OPTS=0 disables oneDNN optimizations for better training stability on some hardware setups.

Image fixing utility

```
def fix_images(folder):
    for subdir, _, files in os.walk(folder):
        for file in files:
            file_path = os.path.join(subdir, file)
            try:
                with Image.open(file_path) as img:
                    img = img.convert('RGB')
                    new_path = os.path.splitext(file_path)[0] + ".jpg"
                    img.save(new_path, 'JPEG')
                    if new_path != file_path:
                        os.remove(file_path)
            except Exception as e:
                print(f"Nie udało się naprawić: {file_path}, błąd: {e}")
                os.remove(file_path)
```

This utility walks through all subfolders and converts images into a clean JPEG format. Corrupted or incompatible files are removed to ensure that all training data is valid.

Training history plots

```python
def model_acc_plt(history_dict):
    num_epochs = len(history_dict['accuracy'])
    epochs = range(1, num_epochs + 1)

    fg, ax = plt.subplots(layout='constrained')
    ax.plot(epochs, history_dict['accuracy'], label='train accuracy')
    ax.plot(epochs, history_dict['val_accuracy'], label='val accuracy')
    ax.xaxis.set_major_locator(MaxNLocator(integer=True))
    ax.set_xlim(1, num_epochs)
    ax.set_xlabel('Epoch')
    ax.set_ylabel('Accuracy')
    ax.set_title('Model Accuracy')
    ax.legend()
    plt.show()


def model_loss_plt(history_dict):
    num_epochs = len(history_dict['loss'])
    epochs = range(1, num_epochs + 1)

    fg, ax = plt.subplots(layout='constrained')
    ax.plot(epochs, history_dict['loss'], label='train loss')
    ax.plot(epochs, history_dict['val_loss'], label='val loss')
    ax.xaxis.set_major_locator(MaxNLocator(integer=True))
    ax.set_xlim(1, num_epochs)
    ax.set_xlabel('Epoch')
    ax.set_ylabel('Loss')
    ax.set_title('Model Loss')
    ax.legend()
    plt.show()
```

Two helper functions to visualize training progress. They plot training and validation accuracy/loss per epoch. This allows you to easily spot overfitting, underfitting, or convergence behavior.

Confusion matrix utility

```
def conf_matrix(y_true, y_pred, class_labels_show):
    cm=confusion_matrix(y_true,y_pred)
    plt.figure(figsize=(14,10))
    sns.heatmap(cm,annot=True,cmap='Blues',fmt='.0f')
    plt.ylabel("True values",size=15)
    plt.xlabel('Predicted values',size=15)
    plt.xticks(ticks=np.arange(len(class_labels_show))+0.5,labels=class_labels_show,rotation=60)
    plt.yticks(ticks=np.arange(len(class_labels_show))+0.5,labels=class_labels_show,rotation=0)
    plt.show()
```

This function computes a confusion matrix and renders it as a heatmap. The axes are labeled with the corresponding class names for better interpretability of the model's predictions vs. true labels.

Prediction for a single image

```
def predict_image(image_path, model, class_labels):
    from tensorflow.keras.preprocessing import image

    img = image.load_img(image_path, target_size=(224, 224))

    img_array = image.img_to_array(img)

    img_array = img_array / 255.0

    img_batch = np.expand_dims(img_array, axis=0)

    predictions = model.predict(img_batch)
    predicted_index = np.argmax(predictions)
    predicted_label = class_labels[predicted_index]
    confidence = predictions[0][predicted_index]

    #print(f"Predicted: {predicted_label} ({confidence * 100:.2f}%)")
    return predicted_label, confidence
```

This helper loads a single image, preprocesses it (resizes, scales), and returns the model's predicted label and its associated confidence score.

Showing a single image prediction

```python
def show_prediction(image_path, model, class_labels):
    label, confidence = predict_image(image_path, model, class_labels)

    img = Image.open(image_path)
    plt.imshow(img)
    plt.title(f"{label} ({confidence*100:.1f}%)")
    plt.axis('off')
    plt.show()
```

This helper function simplifies inspecting a single image.
It uses the predict_image() helper to get the predicted label and its confidence.
It then renders the image with Matplotlib and displays the top predicted label with the associated percentage.
This is useful for quickly verifying the model's behavior on any individual image. It can be done via terminal. User can provide a certain path to an image he want to verify and choose which model will validate it.

Example result:

Building and training the model

```
DATA_DIR = "Formula One Cars"
IMAGE_SIZE = (224, 224)
BATCH_SIZE = 32
#fix_images(DATA_DIR)
datagen = ImageDataGenerator(validation_split=0.2, rescale=1./255)

train_gen = datagen.flow_from_directory(
    DATA_DIR,
    target_size=IMAGE_SIZE,
    batch_size=BATCH_SIZE,
    class_mode='categorical',
    subset='training'
)
class_names = train_gen.class_indices


val_gen = datagen.flow_from_directory(
    DATA_DIR,
    target_size=IMAGE_SIZE,
    batch_size=BATCH_SIZE,
    class_mode='categorical',
    subset='validation'
)

val_gen_pred = datagen.flow_from_directory(
    DATA_DIR,
    target_size=IMAGE_SIZE,
    batch_size=BATCH_SIZE,
    class_mode='categorical',
    subset='validation',
    shuffle=False
)
```

We use ImageDataGenerator with a validation split of 20% and rescale images to [0, 1]. The training and validation generators automatically load images from the directory and encode class labels.

Base model + classification head:

```
inputs = layers.Input(shape=(224, 224, 3), name='input_layer')

base_model = MobileNetV2(include_top=False)
base_model.trainable = False

x = base_model(inputs, training=False)
x=layers.GlobalAveragePooling2D()(x)
num_classes=len(class_names)
outputs=layers.Dense(num_classes,activation='softmax',dtype=tf.float32)(x)

model = Model(inputs, outputs)
```

Model 1 - MobileNetV2

```
inputs = layers.Input(shape=(224, 224, 3), name='input_layer')

base_model = ResNet50V2(include_top=False, classes=len(class_names))
base_model.trainable = False

x = base_model(inputs, training=False)
x=layers.GlobalAveragePooling2D()(x)
num_classes=len(class_names)
outputs=layers.Dense(num_classes,activation='softmax',dtype=tf.float32)(x)

model = Model(inputs, outputs)
```

Model 2 - ResNet50V2

```
inputs = layers.Input(shape=(224, 224, 3), name='input_layer')

base_model = NasNetMobile(include_top=False, classes=len(class_names))
base_model.trainable = False

x = base_model(inputs, training=False)
x=layers.GlobalAveragePooling2D()(x)
num_classes=len(class_names)
outputs=layers.Dense(num_classes,activation='softmax',dtype=tf.float32)(x)

model = Model(inputs, outputs)
```

Model 3 – NasNetMobile

In this project, we experimented with several pre-trained models (MobileNetV2, NASNetMobile, and ResNet50V2) as the backbone for transfer learning. MobileNetV2 was ultimately selected because it provided the best trade-off between accuracy, training time, and model size. The final architecture uses MobileNetV2 for feature extraction and a simple dense classifier head with a softmax output. Initially, the MobileNetV2 layers are frozen for efficient training.

Training and fine-tuning:

```
model.compile(
    optimizer='adam',
     loss='categorical_crossentropy',
    metrics=['accuracy']
)

model.fit(train_gen, validation_data=val_gen, epochs=5)
model.save("model/f1_model_MobileNetV2afterRT.h5")

#fine-tuning
base_model.trainable = True
model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=1e-5),
    loss='categorical_crossentropy',
    metrics=['accuracy']
)
model.fit(train_gen, validation_data=val_gen, epochs=5)
model.save("model/f1_model_fine_tuned_MobileNetV2afterRT.h5")
```

Training is performed in two stages:

1. **Feature extraction:** Train the top classifier while the MobileNetV2 base is frozen.
2. **Fine-tuning:** Unfreeze the base and train all layers at a very small learning rate for fine-grained adjustments.

Saving the model and training history

```
#save data for later
history_dict=model.history.history
json.dump(history_dict, open('model/history_dict_MobileNetV2afterRT.json', 'w'))

trained_model = load_model("model/f1_model_MobileNetV2afterRT.h5")
trained_model.summary()
```

The trained model and its training history are saved to disk. The JSON file contains accuracy and loss history so that we can easily visualize the training curves later without retraining.

5. Our Network – Code

Preparing the Dataset and Model

```
DATA_DIR = "Formula One Cars"
IMAGE_SIZE = (224, 224)
BATCH_SIZE = 32
datagen = ImageDataGenerator(validation_split=0.2, rescale=1./255)

train_gen = tf.keras.utils.image_dataset_from_directory(
    DATA_DIR,
    validation_split=0.2,
    subset="training",
    seed=123,
    image_size=(224, 224),
    batch_size=32)

class_names = train_gen.class_names


val_gen = tf.keras.utils.image_dataset_from_directory(
    DATA_DIR,
    validation_split=0.2,
    subset="validation",
    seed=123,
    image_size=(224, 224),
    batch_size=32)

val_gen_pred = tf.keras.utils.image_dataset_from_directory(
    DATA_DIR,
    validation_split=0.2,
    subset="validation",
    seed=123,
    image_size=(224, 224),
    batch_size=32,
    shuffle = False)

num_classes=len(class_names)
```

Here we prepare the training and validation datasets:
We split the images into training and validation sets (80% training / 20% validation).
Images are loaded at 224 x 224 resolution.
The validation set is also prepared separately (val_gen_pred) for later predictions and evaluation.

Building the CNN Model From Scratch

```
model = models.Sequential([
    layers.Conv2D(16, 3, padding='same', activation='relu'),
    layers.BatchNormalization(),
    layers.MaxPooling2D(),

    layers.Conv2D(32, 3, padding='same', activation='relu'),
    layers.BatchNormalization(),
    layers.MaxPooling2D(),

    layers.Conv2D(64, 3, padding='same', activation='relu'),
    layers.BatchNormalization(),
    layers.MaxPooling2D((2, 2)),

    layers.Conv2D(128, 3, padding='same', activation='relu'),
    layers.BatchNormalization(),
    layers.MaxPooling2D(),

    layers.GlobalAveragePooling2D(),
    layers.Dense(224, activation='relu'),
    layers.Dropout(0.5),
    layers.Dense(num_classes, activation='softmax')
])

model.compile(optimizer=tf.keras.optimizers.Adam(learning_rate=1e-3),
              loss='categorical_crossentropy',
              metrics=['accuracy'])
model.summary()
```

Here we construct a simple Convolutional Neural Network:
Several Conv2D + MaxPooling2D blocks extract image features.
A dense layer (Dense(224, relu)) is placed before the final classification layer. The dropout prevents the model from overfitting.
Finally, we compile the model using Adam and sparse categorical cross-entropy as the loss.

Training the Model and Saving the History

```
history = model.fit(train_gen, validation_data=val_gen, epochs=5)
model.save("model/my_f1_model.h5")

history_dict = model.history.history
json.dump(history_dict, open('model/my_model_history_dict.json', 'w'))
```

Train the model for 5 epochs.
Save the trained model and its training history (history_dict) as JSON for later plotting.

Evaluating the Model

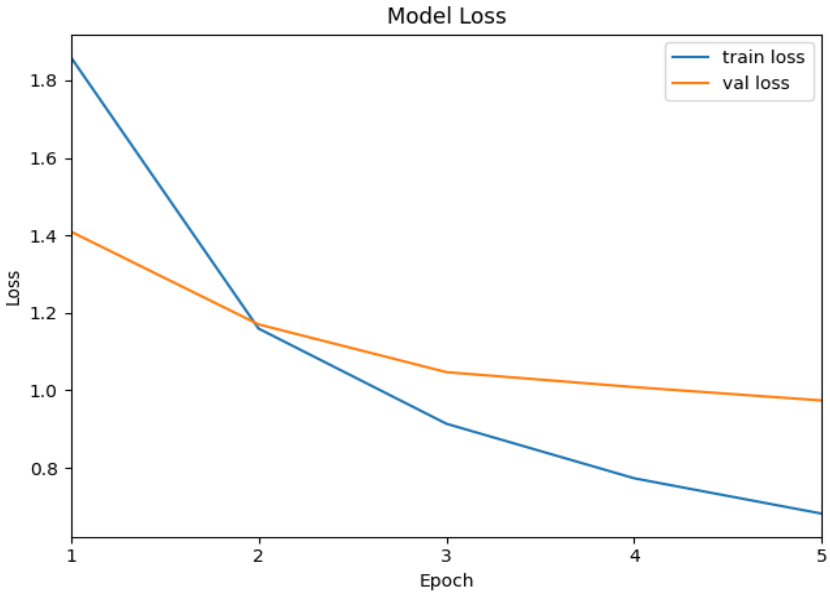
```
y_pred_probs = model.predict(val_gen_pred)
y_pred = y_pred = np.argmax(y_pred_probs, axis=1)
y_true = val_gen_pred.class_names
class_labels = list(val_gen_pred.class_names)

model_acc_plt(history_dict)
model_loss_plt(history_dict)
conf_matrix(y_true, y_pred, class_labels)
```

Generate predictions on the validation set and extract the predicted classes.
Compute the confusion matrix.
Plot the training and validation accuracy and loss over the training process using helper functions (model_acc_plt, model_loss_plt), and visualize the confusion matrix.

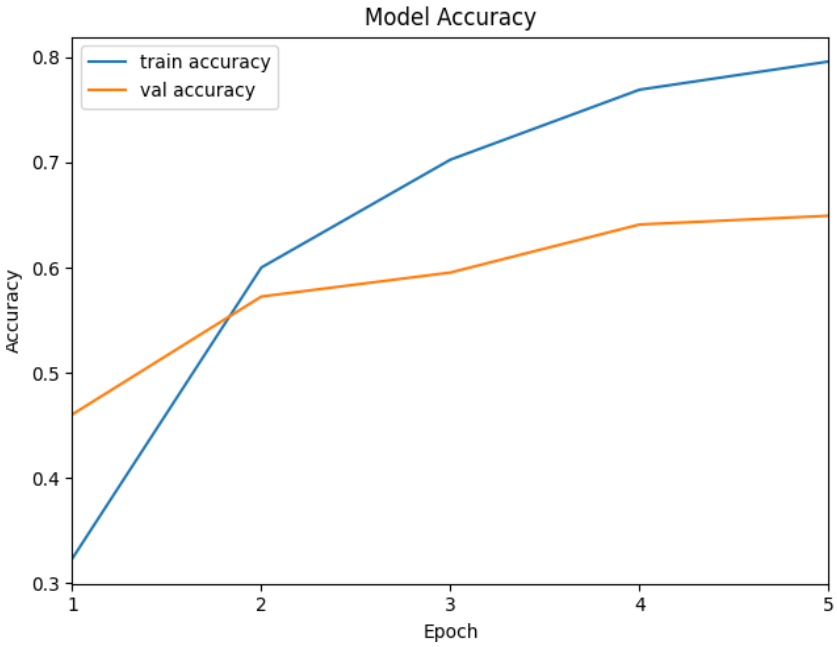
6. Experiments

First, we experimented with various pre-trained models, to see which one will do best.

a) ResNet50V2 before Fine-Tuning

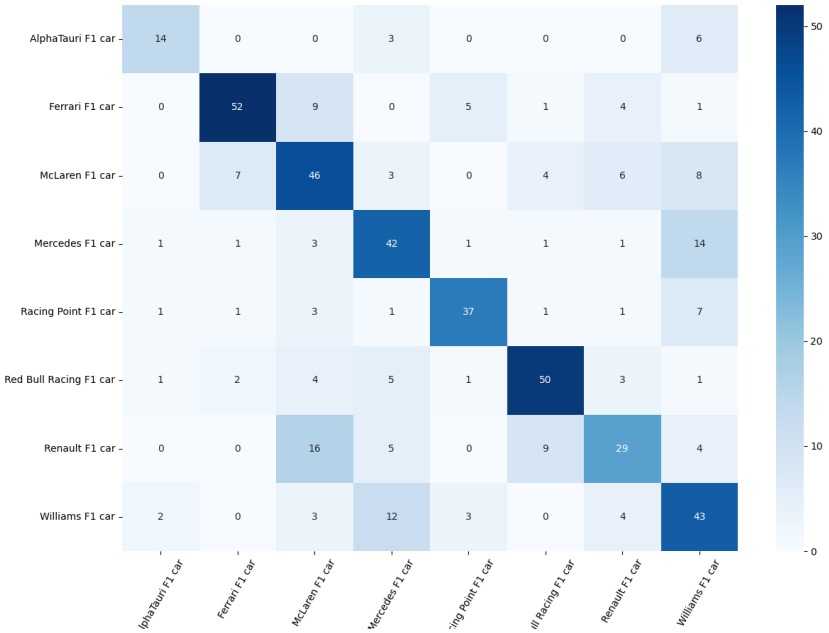


Model Loss - Displays how the training and validation loss (error) changed over epochs, The downward trend indicates that the model is learning and improving – making fewer errors, Since the training and validation loss curves stay relatively close, it suggests the model is not overfitting.

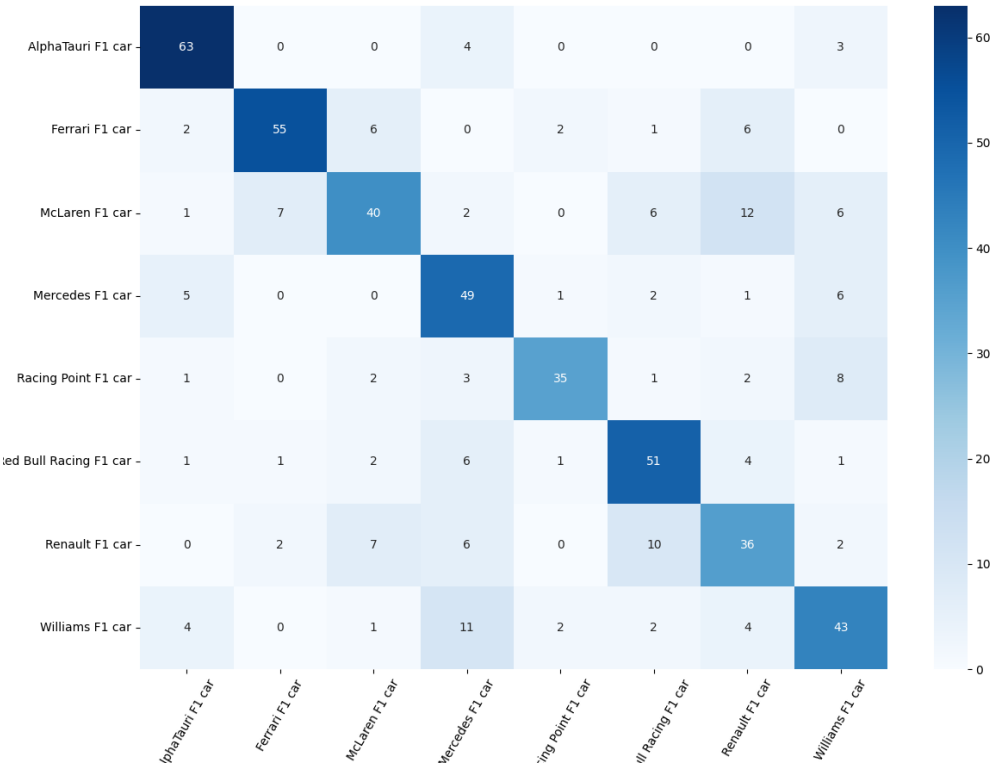


Model Accuracy - Shows how classification accuracy improves over time for both training and validation sets, Accuracy steadily increases, indicating that the model continues to learn effectively each epoch, The validation accuracy follows a similar trend, meaning the model is generalizing well to unseen data.

Confusion matrix before data augmentation (AlphaTauri)

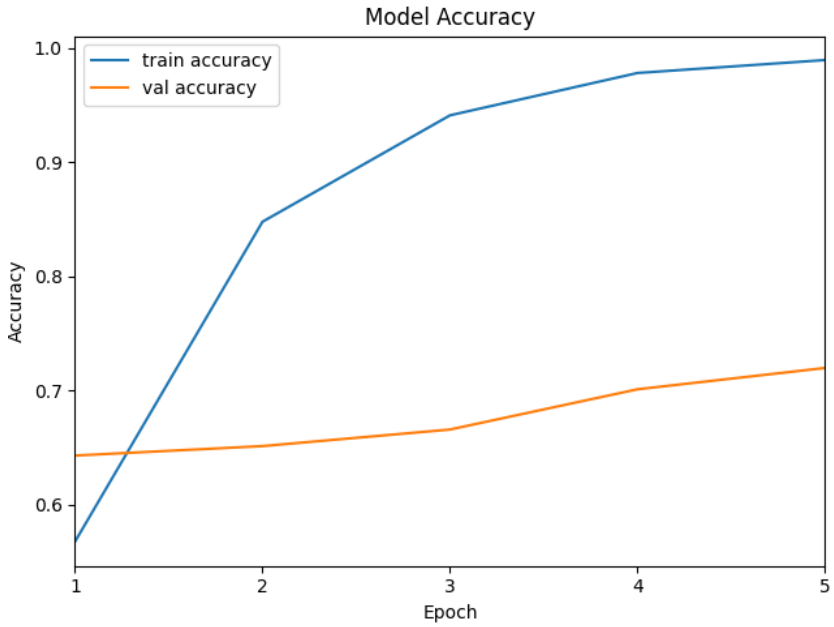


Confusion matrix after data augmentation (AlphaTauri)



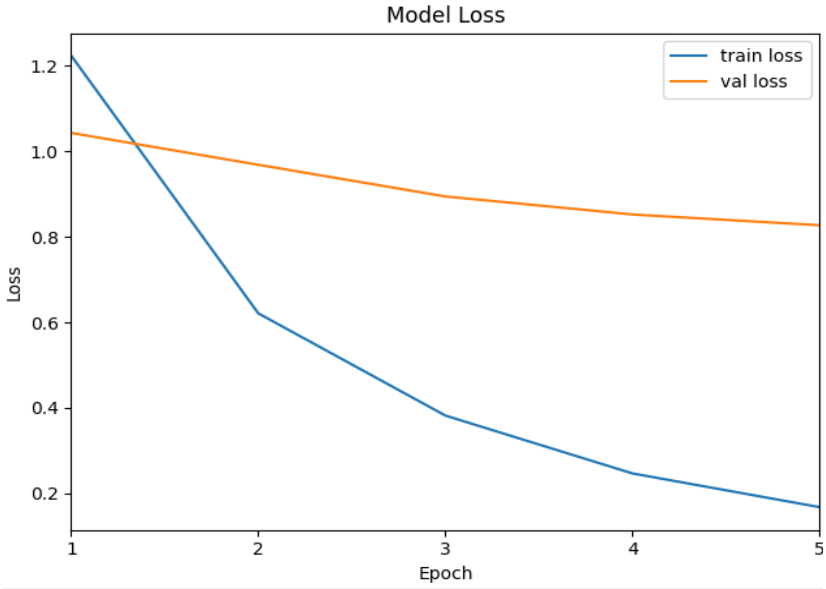
Data augmentation is a technique where we artificially expand the training dataset by creating modified versions of the original images. This can include random transformations like rotations, flips, zooms, shifts, or changes in brightness and contrast.

ResNet50V2 after Fine_Tuning



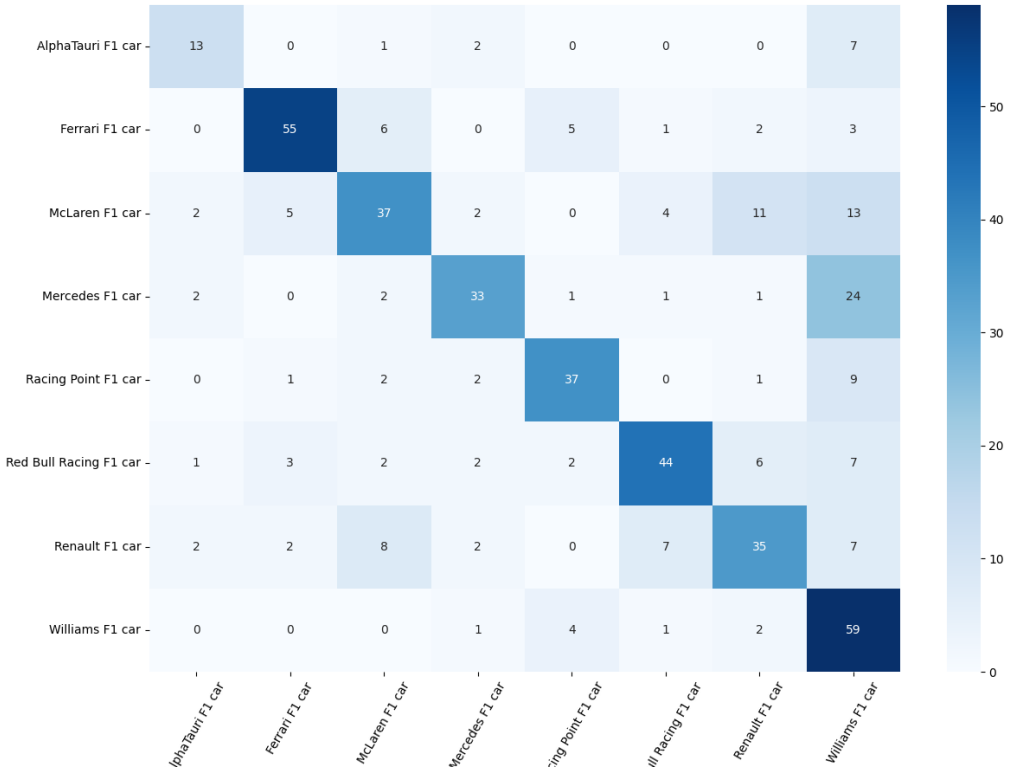
Shows accuracy over epochs for both training and validation sets, Training accuracy increases rapidly, reaching nearly 100%, which indicates that the model has fully fit the

training data, Validation accuracy improves more slowly, suggesting that fine-tuning helped, but the gap between train and validation accuracy implies possible overfitting.



Displays a steep decline in training loss, which aligns with the strong training accuracy, Validation loss decreases gradually, confirming slight generalization improvements, However, the growing gap between training and validation loss may indicate diminishing returns from continued training without regularization.

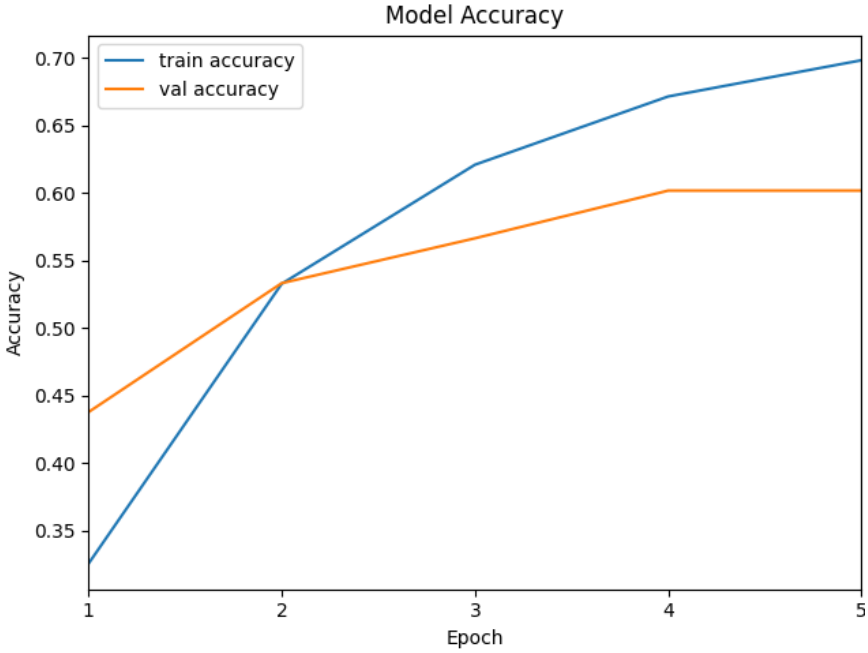
Confusion matrix before data augmentation



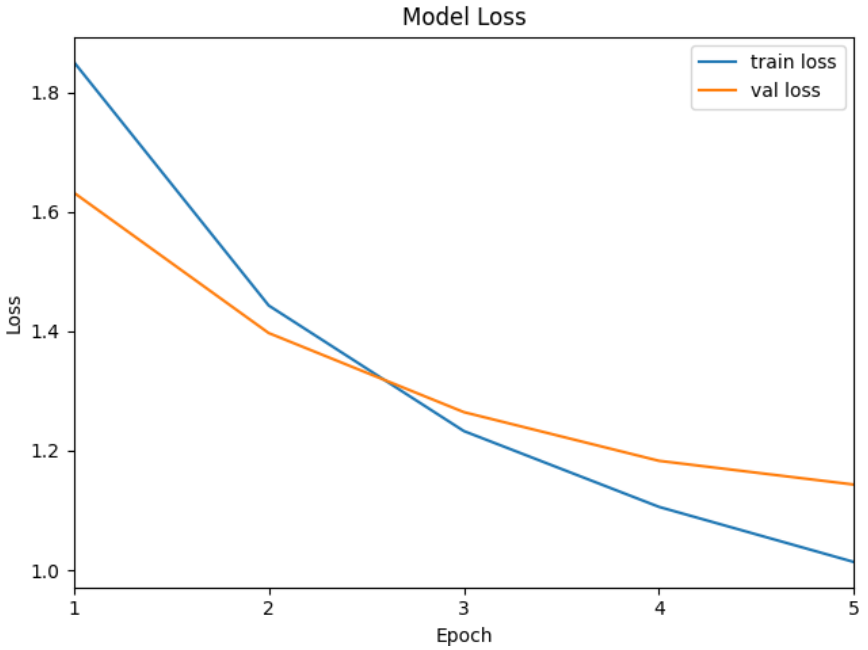
Confusion matrix in this model came out relatively well, especially after data augmentation. The result can be satisfactory, considering that this model is quite large and slow to interpret

Before fine-tuning, we used data augmentation to increase the number of images in one class from about 100 to 350, balancing the dataset. However, the fine-tuning was still done on the original, smaller set, allowing the model to learn more generalizable features without overfitting to artificially augmented data.

b) NasNetMobile before Fine-Tuning

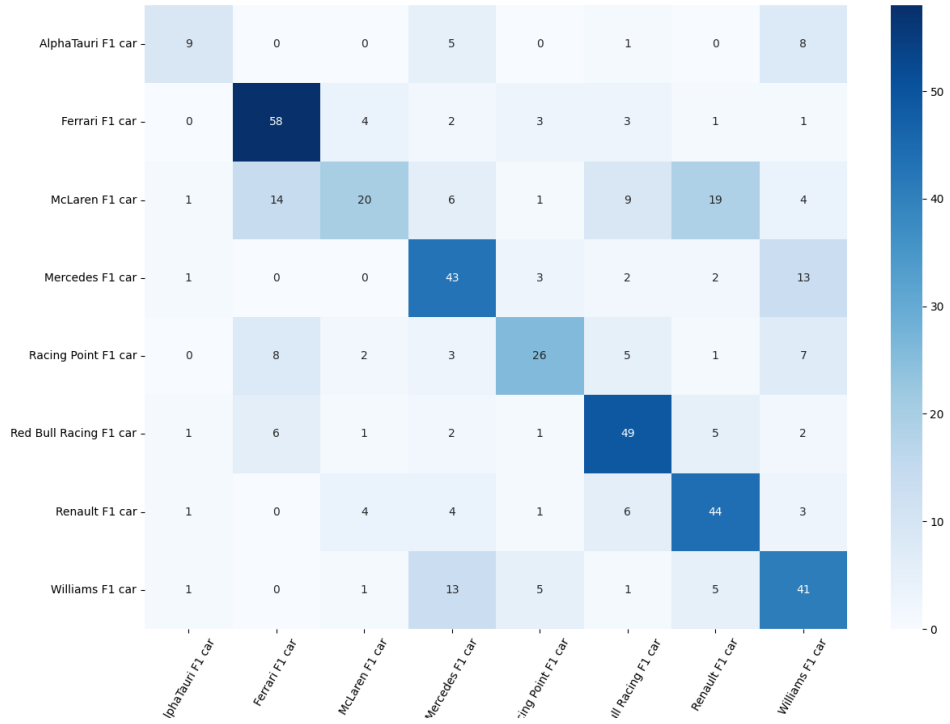


Training accuracy increases steadily, starting from ~35% and reaching ~70%, Validation accuracy also improves, but levels off around ~60%, The similar trends between training and validation accuracies suggest the model is learning without overfitting.

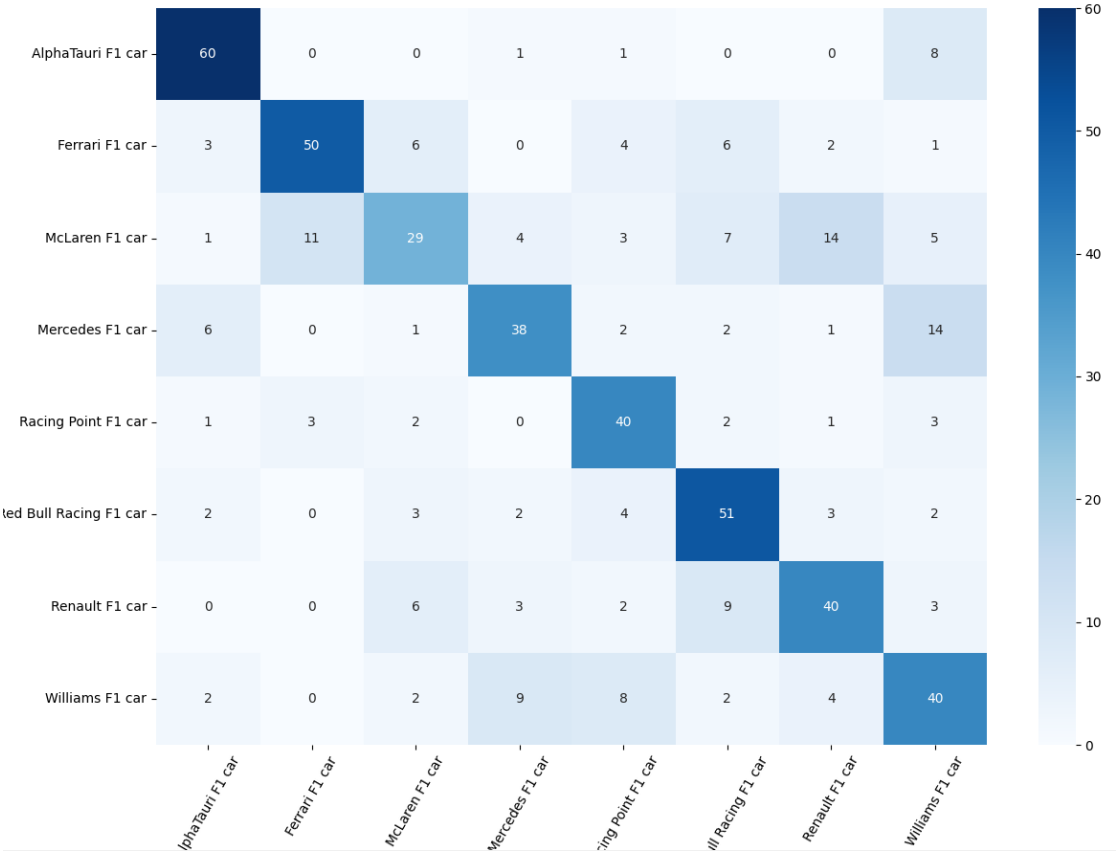


Both training and validation loss decrease, which is a good sign that the model is converging, The gap between training and validation loss is small, indicating good generalization and no major overfitting at this stage.

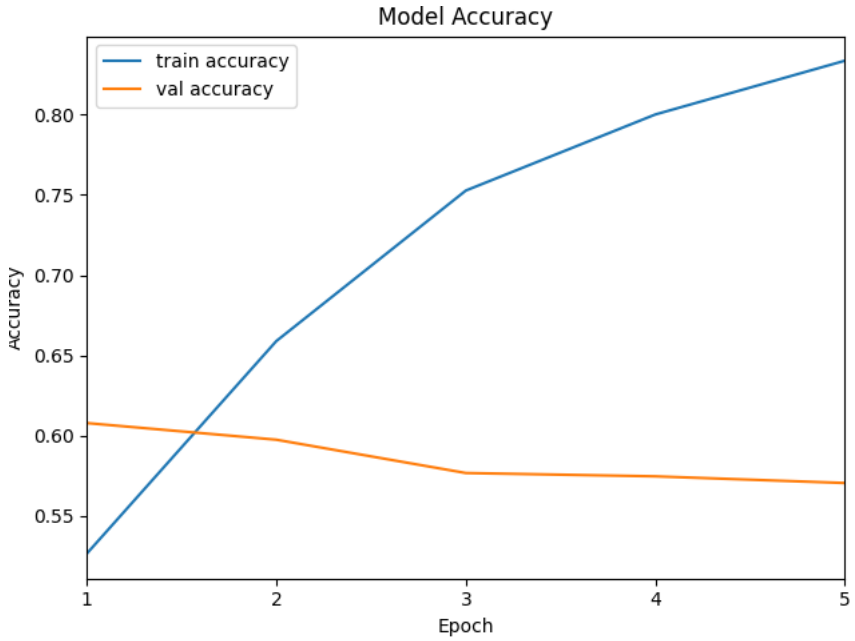
Confusion matrix before data augmentation (AlphaTauri)



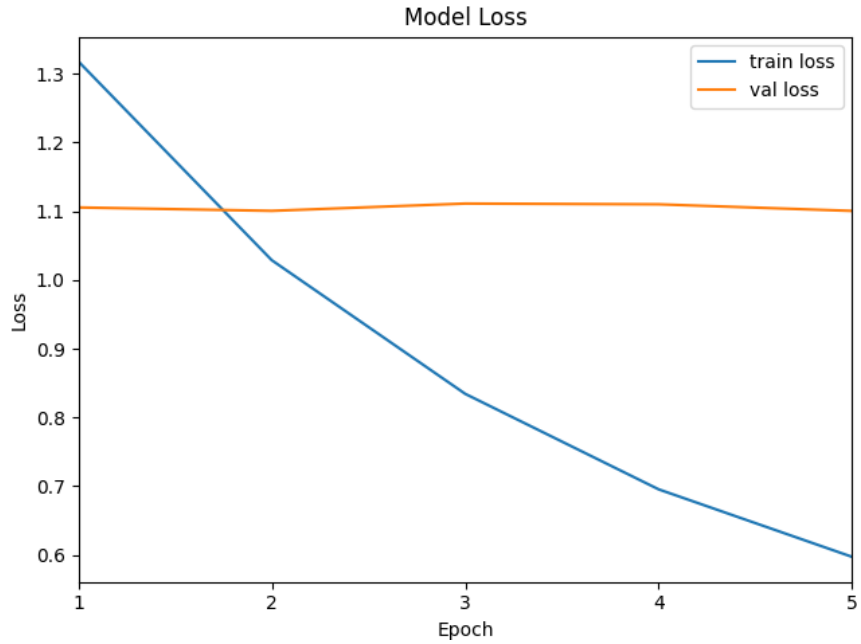
Confusion matrix after data augmentation (AlphaTauri)



NasNetMobile after Fine-Tuning

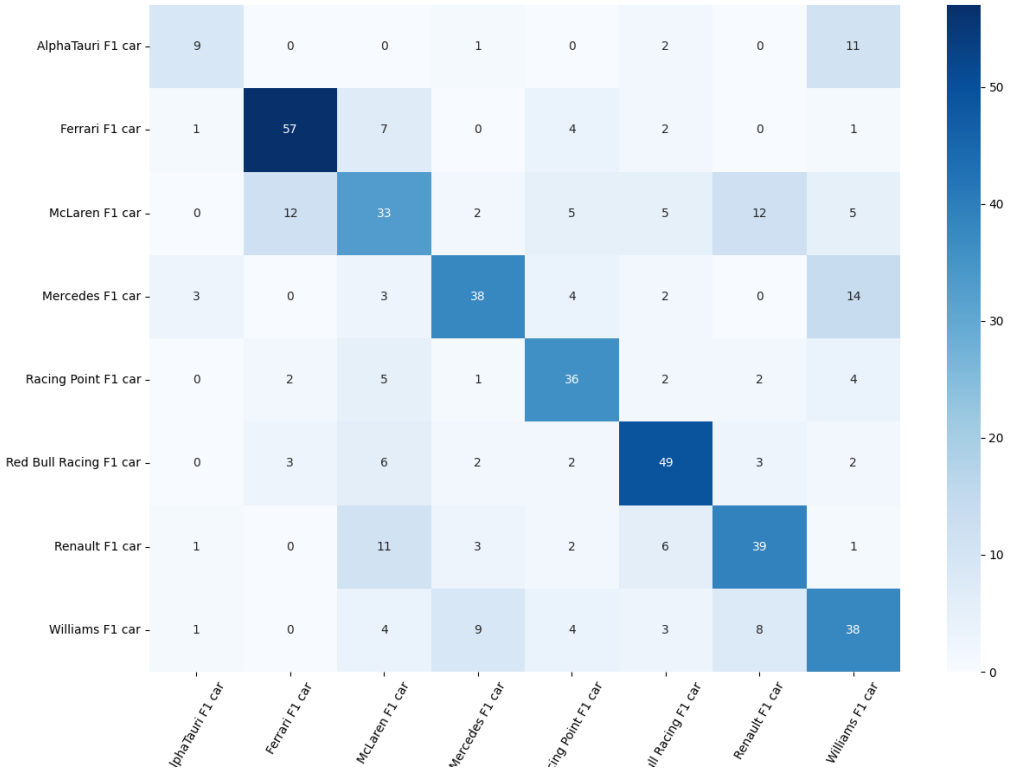


Training accuracy improves significantly, rising from ~53% to ~86%, Validation accuracy, on the other hand, slightly decreases from ~61% to ~57%, The growing gap between training and validation accuracy suggests overfitting – the model memorizes training data but struggles with generalization.



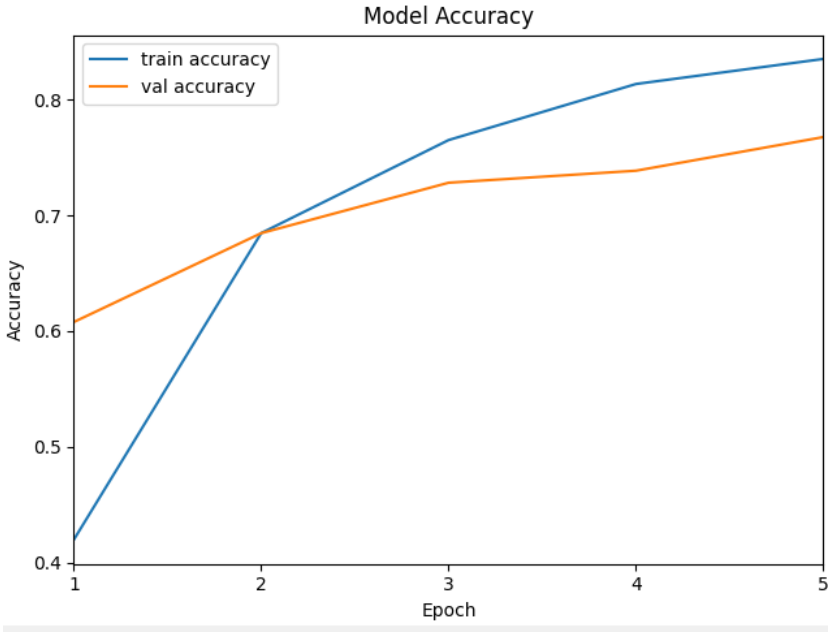
Training loss decreases steadily, from ~1.3 to ~0.6, Validation loss remains relatively constant, around ~1.1, This consistent difference between training and validation loss confirms the overfitting observed in the accuracy plot – the model is not learning effectively from validation data.

Confusion matrix before data augmentation

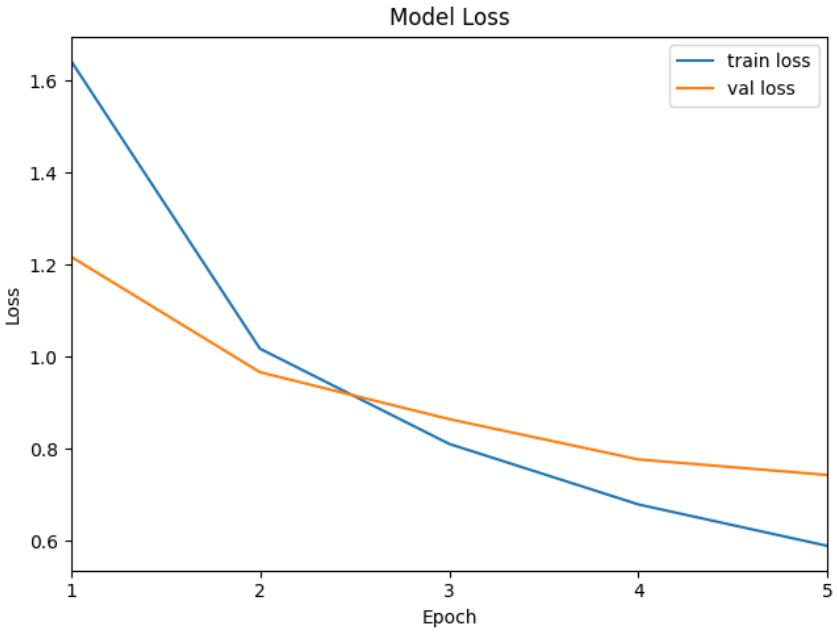


In this case, the confusion matrix came out the worst. This may be due to the fact that the architecture is more complicated to implement and slightly heavier computationally.

c) **MobileNetV2 before Fine-Tuning**

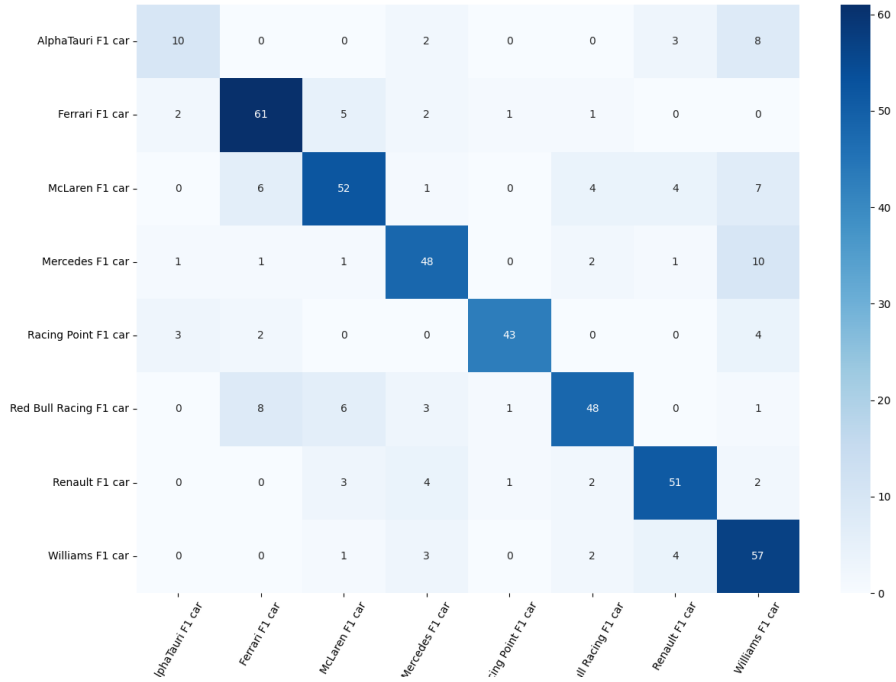


This plot shows how the model's accuracy improves over 5 epochs.
Training accuracy increases significantly from ~42% to ~84%, Validation accuracy also improves steadily, starting at ~61% and reaching ~77%,
The similar upward trend in both curves suggests good generalization and no sign of overfitting at this stage.

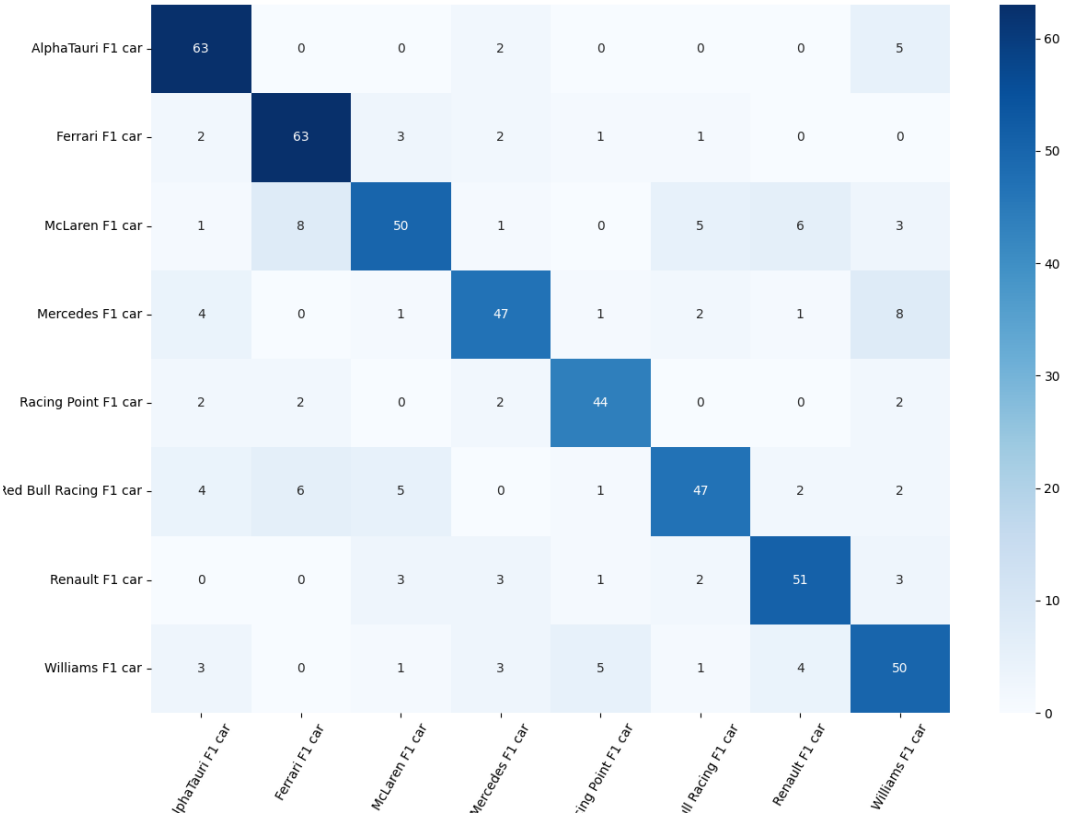


Both training and validation loss decrease clearly over time.
Training loss drops from ~1.65 to ~0.58, while validation loss decreases from ~1.2 to ~0.73, The gap between training and validation loss remains small, indicating that the model is learning effectively and generalizing well before fine-tuning.

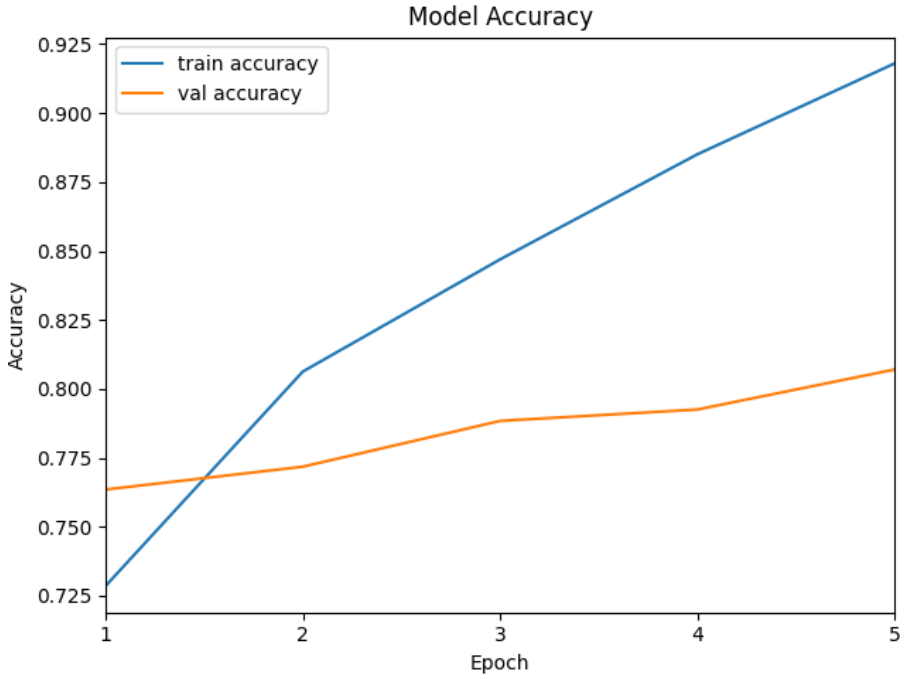
Confusion matrix before data augmentation (AlphaTauri)



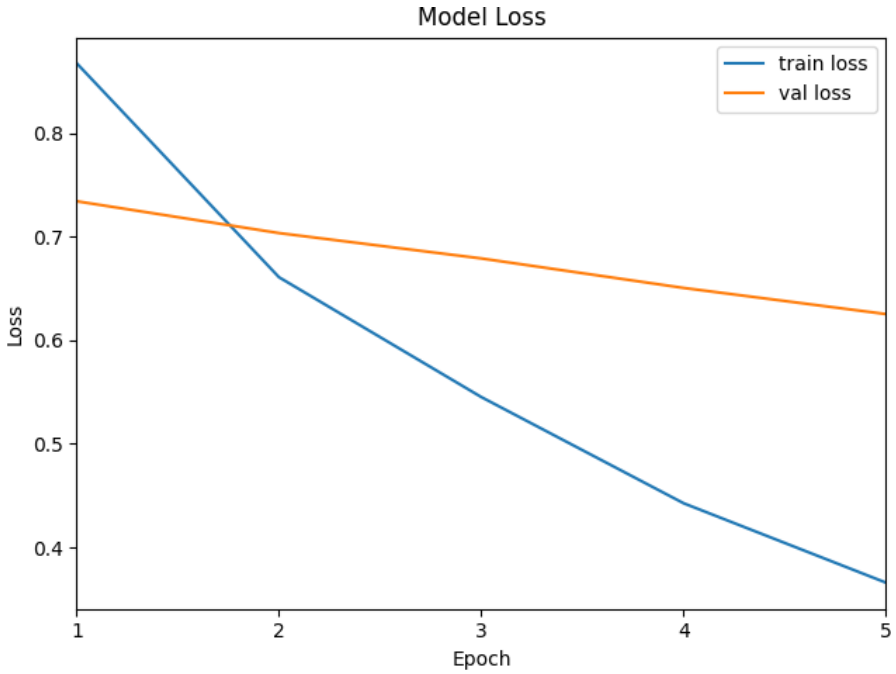
Confusion matrix after data augmentation (AlphaTauri)



MobileNetV2 after Fine-Tuning

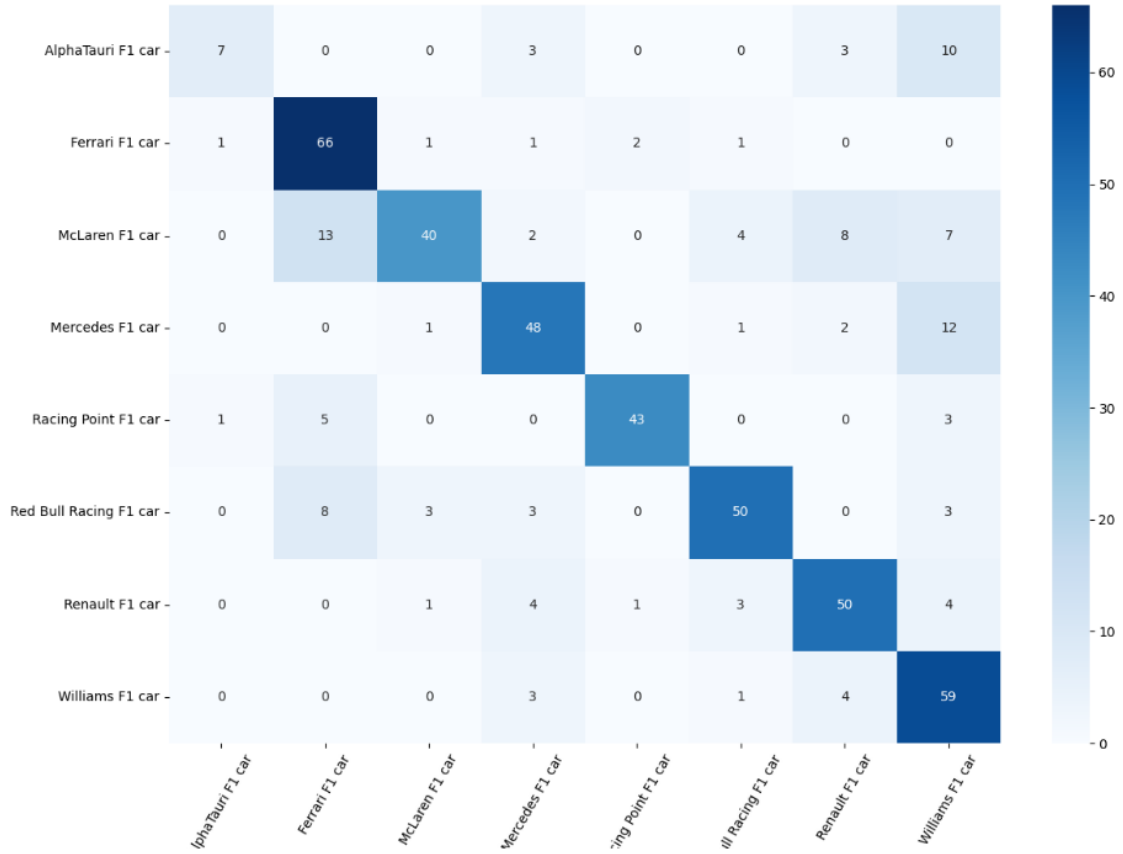


Training accuracy rises from ~72% to ~92%.
Validation accuracy also increases, going from ~76% to ~81%, Both curves continue to improve, and the gap between them remains moderate, suggesting the model benefits from fine-tuning and still maintains good generalization.



Training loss decreases sharply from ~0.87 to ~0.37, Validation loss also declines gradually from ~0.74 to ~0.63, The loss curves are close and trend downward, which confirms effective learning and no significant overfitting after fine-tuning.

Confusion matrix before data augmentation



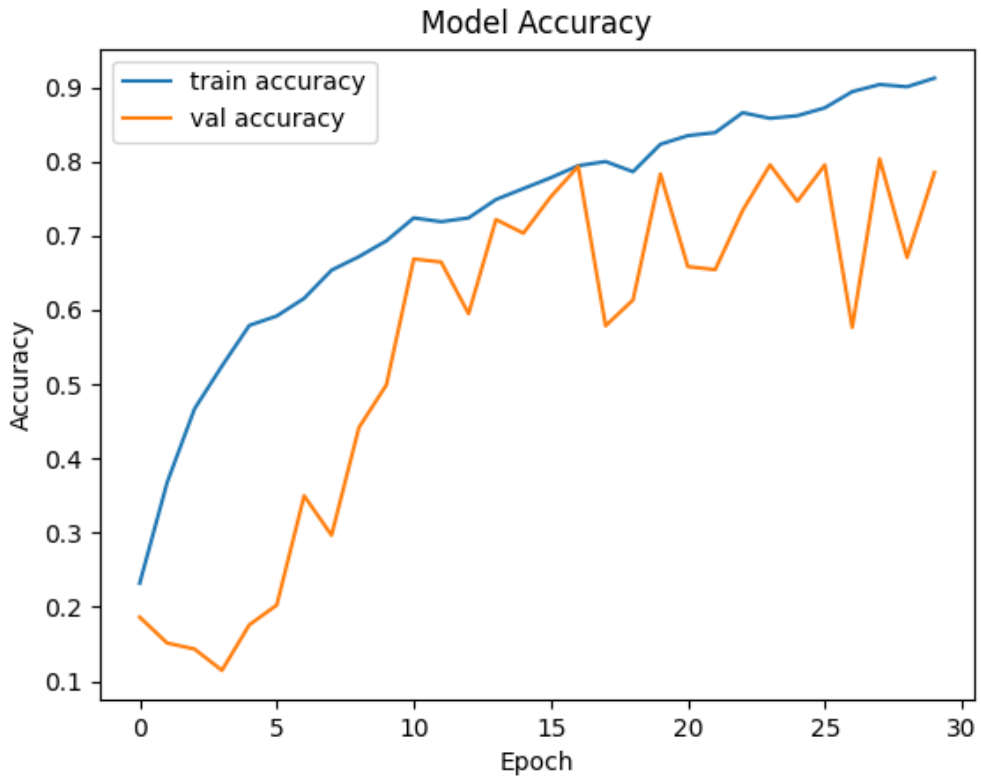
This model performed the best. It is fast in inference and small in memory and has very good accuracy compared to its size.

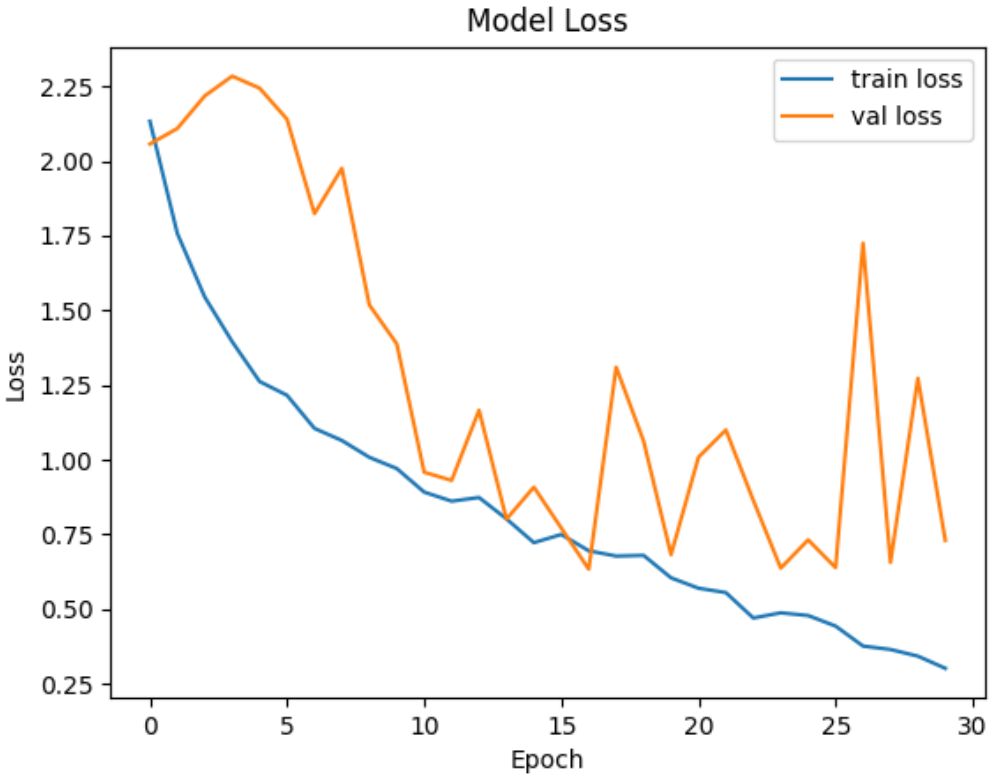
d) Our model

Since our model was built from scratch, we had to train it for a significantly higher number of epochs. Our dataset was relatively small for a model learning from scratch, so we introduced several modifications to help it learn to recognize Formula 1 cars effectively:

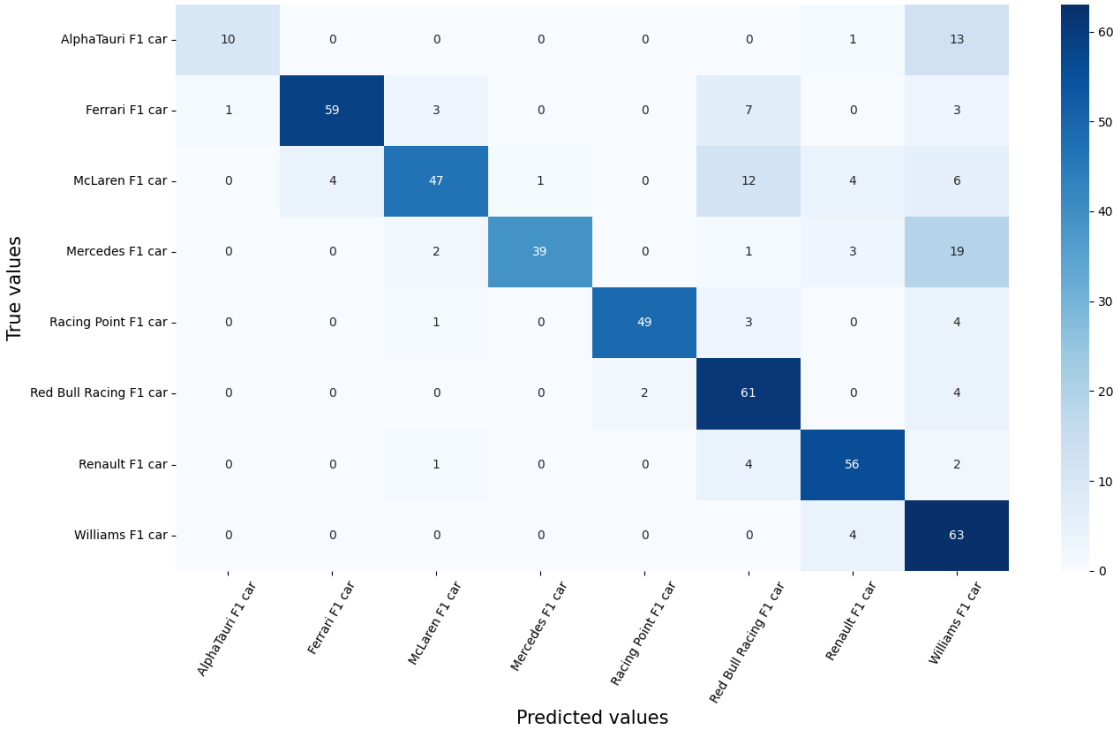
- Dropout – adding the dropout prevented the model from over-fitting, which was very likely to happen, because of the small dataset
- We had to use categorical_crossentropy loss with combination with softmax, without this setup, model wasn't learning properly.
- Increasing the learning rate in optimizer increased the accuracy of training
- Batch Normalization – stabilized and accelerated the training process

These strategies allowed us to build custom CNN neural network that achieved meaningful performance.





Confusion matrix before data augmentation



Why didn’t some models work?

- Some models are designed for large datasets
- They can be sensitive to the learning rate
- Images were not properly scaled
- They required more advanced preprocessing and fine-tuning

7. Conclusions

Fine-tuning significantly improved the model’s learning ability (the model learned to recognize patterns more accurately), but it also increased the risk of overfitting. It also made learning a lot longer.
Classes with similar appearances or a similar number of examples are harder to distinguish.
Instead of training a network from scratch, using pre-trained architectures like MobileNetV2, NASNetMobile, or ResNet50V2 allows to train faster and reach high accuracy with much less data.
Having a large and diverse dataset helps the model generalize better to new, unseen images.
When the dataset is small, applying data augmentation techniques — like rotations, flips, zooms, and color changes — artificially increases the dataset size and variety, reducing overfitting.